PYTHON ACTIVITY-BASED LEARNING

Clear explanations, algorithms, flowcharts with standard symbols, pseudocode, Python programs and sample outputs

Prepared from the uploaded 22-page activity document. The original rules, values and case-study order are retained. Each flowchart is written using symbol labels so it can be easily copied into a hand-drawn diagram.

Flowchart symbols used

Symbol / shape	Meaning	Use in these programs
Oval	Start / End	Beginning and stopping the program
Parallelogram	Input / Output	Read values or display results
Rectangle	Process	Calculations and assignments
Diamond	Decision	Yes/No conditions such as valid input or eligibility
Arrow	Flow line	Shows the direction of execution

How to read a flowchart: Follow the arrows from START. At a diamond, choose the YES or NO branch. A rectangle means perform a calculation. A parallelogram means input or output.

1. University Scholarship Allocation

Problem statement

Check scholarship applicants using aggregate marks, minimum subject marks, attendance and annual family income. Eligible applicants are ranked by aggregate marks, then attendance.

Objectives

• Check all four eligibility conditions.
• Store applicant details as tuples inside a list.
• Rank eligible applicants; attendance breaks a tie in aggregate marks.

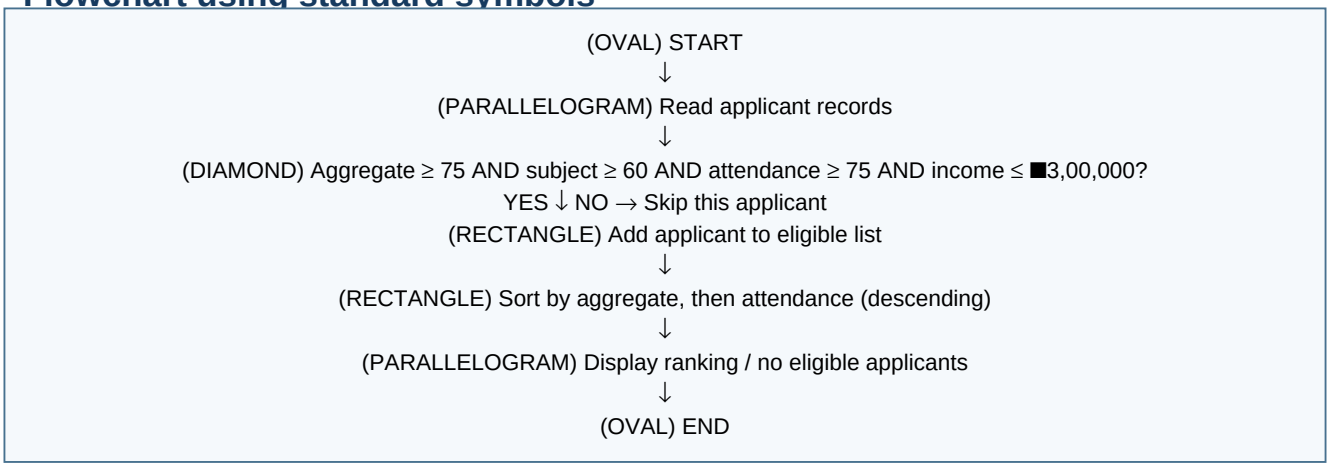
Assumptions / rules

• Aggregate marks ≥ 75%.
• Minimum subject mark ≥ 60%.
• Attendance ≥ 75%.
• Annual family income ≤ ■3,00,000.
• Higher attendance wins a tie.

Algorithm (step by step)

1. Start and store applicant records.
2. For each applicant, check marks, subject minimum, attendance and income.
3. If all conditions are true, add the applicant to the eligible list.
4. Sort eligible applicants by aggregate marks (high to low), then attendance (high to low).
5. Display the ranked list, or a no-eligible message.
6. Stop.

Flowchart using standard symbols



How the program works

The program checks every applicant separately. An applicant is eligible only when all four conditions are satisfied. The sort key uses aggregate marks first and attendance second, both in descending order.

Python source code

```
applicants = [
    ("Anu", 88, 70, 90, 250000),
    ("Bala", 78, 62, 80, 280000),
    ("Charan", 91, 55, 95, 200000),
    ("Divya", 85, 68, 72, 180000)
]

eligible = []

for name, aggregate, subject_min, attendance, income in applicants:
    if (aggregate >= 75 and subject_min >= 60
```

```
            and attendance >= 75 and income <= 300000):
        eligible.append((name, aggregate, attendance))

eligible.sort(key=lambda student: (student[1], student[2]), reverse=True)

print("Eligible Scholarship Applicants")
if eligible:
    for rank, student in enumerate(eligible, start=1):
        print(rank, student[0], "- Aggregate:", student[1],
              "Attendance:", str(student[2]) + "%")
else:
    print("No eligible applicants")
```

Sample output

```
Eligible Scholarship Applicants
1 Anu - Aggregate: 88 Attendance: 90%
2 Bala - Aggregate: 78 Attendance: 80%
```

Test cases to understand

- Anu: eligible; ranked first.
- Charan: not eligible because subject minimum is 55.
- Divya: not eligible because attendance is 72%.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

2. Progressive Electricity Billing

Problem statement

Calculate a household electricity bill using three consumption slabs, a fixed charge, a high-bill surcharge and a low-consumption rebate.

Objectives

• Calculate energy charges slab by slab.
• Add the fixed charge and apply the correct adjustment.
• Display the bill breakdown.

Assumptions / rules

• First 100 units: ■1.50 per unit.
• Units 101–200: ■2.50 per unit.
• Units above 200: ■4.00 per unit.
• Fixed charge: ■50.
• If bill before adjustment > ■500, add 5% surcharge. Otherwise, consumption up to 100 units receives a 10% rebate on energy charges.

Algorithm (step by step)

1. Read units consumed.
2. If units are negative, display invalid units.
3. Calculate energy charge using the applicable slabs.
4. Add ■50 fixed charge.
5. If the bill is above ■500, add 5%; otherwise, if units ≤ 100, subtract 10% of energy charge.
6. Display energy charge, adjustment and final bill.
7. Stop.

Flowchart using standard symbols

(OVAL) START
↓
(PARALLELOGRAM) Input units consumed
↓
(DIAMOND) Units ≥ 0? — NO → (PARALLELOGRAM) Display invalid units → END
YES ↓
(DIAMOND) Units ≤ 100? → YES: energy = units × 1.50
NO ↓
(DIAMOND) Units ≤ 200? → YES: energy = 150 + (units – 100) × 2.50
NO: energy = 400 + (units – 200) × 4.00
↓
(RECTANGLE) Bill = energy + 50
↓
(DIAMOND) Bill > ■500? → YES: add 5% surcharge
NO: if units ≤ 100, subtract 10% of energy as rebate
↓
(PARALLELOGRAM) Display bill details
↓
(OVAL) END

How the program works

The energy charge is progressive: each block of units is charged at its own rate. The fixed charge is added after calculating energy. The surcharge is checked first; the rebate is used only when the surcharge condition is false and units are 100 or less.

Python source code

```
units = int(input("Enter units consumed: "))

if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy = units * 1.50
    elif units <= 200:
        energy = 100 * 1.50 + (units - 100) * 2.50
    else:
        energy = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00

    bill = energy + 50
    adjustment = 0

    if bill > 500:
        adjustment = bill * 0.05
        bill += adjustment
        print("Surcharge:", round(adjustment, 2))
    elif units <= 100:
        adjustment = energy * 0.10
        bill -= adjustment
        print("Rebate:", round(adjustment, 2))

    print("Energy charge: Rs.", round(energy, 2))
    print("Final bill: Rs.", round(bill, 2))
```

Sample output

```
Enter units consumed: 250
Surcharge: 32.5
Energy charge: Rs. 600.0
Final bill: Rs. 682.5
```

Test cases to understand

• 80 units: energy ■120; rebate ■12; final bill ■158.
• 250 units: energy ■600; surcharge ■32.50; final bill ■682.50.
• –5 units: invalid units.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

3. Hospital Billing and Patient Classification

Problem statement

Prepare a hospital bill by adding consultation, diagnostic and treatment charges. Give senior patients a concession and classify the final bill.

Objectives

• Calculate total service charges.
• Apply a concession when the age condition is met.
• Classify the bill as High Cost or Standard.

Assumptions / rules

• Consultation: ■500; diagnostics: ■800; treatment: ■1,500.
• Age 60 or above receives 10% concession.
• Final bill ≥ ■2,500 is High Cost; otherwise Standard.
• Negative age is invalid.

Algorithm (step by step)

1. Read patient age.
2. If age is negative, display invalid age.
3. Add consultation, diagnostic and treatment charges.
4. If age ≥ 60, calculate 10% concession; otherwise concession is zero.
5. Subtract concession from total.
6. Classify final bill and display details.
7. Stop.

Flowchart using standard symbols

(OVAL) START
↓
(PARALLELOGRAM) Input patient age
↓
(DIAMOND) Age ≥ 0? — NO → Display invalid age → END
YES ↓
(RECTANGLE) Total = 500 + 800 + 1500 = ■2800
↓
(DIAMOND) Age ≥ 60? — YES → Concession = 10% of total; NO → Concession = 0
↓
(RECTANGLE) Final bill = total – concession
↓
(DIAMOND) Final bill ≥ ■2500? — YES: High Cost; NO: Standard
↓
(PARALLELOGRAM) Display bill and category
↓
(OVAL) END

How the program works

The three service charges are added to get ■2,800. A patient aged 60 or more receives a 10% reduction. The category is decided using the final amount after the concession.

Python source code

```
age = int(input("Enter patient age: "))

if age < 0:
    print("Invalid age")
else:
    consultation = 500
```

```
diagnostics = 800
treatment = 1500
total = consultation + diagnostics + treatment

concession = total * 0.10 if age >= 60 else 0
final_bill = total - concession
category = "High Cost" if final_bill >= 2500 else "Standard"

print("Total charges: Rs.", total)
print("Concession: Rs.", round(concession, 2))
print("Final bill: Rs.", round(final_bill, 2))
print("Patient billing category:", category)
```

Sample output

```
Enter patient age: 65
Total charges: Rs. 2800
Concession: Rs. 280.0
Final bill: Rs. 2520.0
Patient billing category: High Cost
```

Test cases to understand

• Age 65: total ■2,800; concession ■280; final ■2,520; High Cost.
• Age 35: final ■2,800; High Cost.
• Age –2: invalid age.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

4. Employee Payroll and Deduction Analysis

Problem statement

Calculate gross salary from basic pay, allowances, overtime and incentives. Subtract provident-fund and progressive tax deductions to find net salary.

Objectives

• Calculate HRA, DA, overtime and gross salary.
• Calculate PF and progressive tax.
• Display a salary statement.

Assumptions / rules

• HRA = 20% of basic pay; DA = 10% of basic pay.
• Overtime rate = ■150 per hour.
• PF = 12% of basic pay.
• Tax = 5% on first ■30,000 of gross salary and 10% on the amount above ■30,000.
• All inputs must be non-negative.

Algorithm (step by step)

1. Read basic pay, overtime hours and incentive.
2. If any input is negative, display invalid input.
3. Calculate HRA, DA and overtime pay.
4. Add basic pay, allowances, overtime and incentive to get gross salary.
5. Calculate PF and progressive tax.
6. Subtract PF and tax from gross salary.
7. Display the salary statement and stop.

Flowchart using standard symbols

(OVAL) START
↓
(PARALLELOGRAM) Input basic pay, overtime hours, incentive
↓
(DIAMOND) Are all inputs ≥ 0? — NO → Display invalid input → END
YES ↓
(RECTANGLE) HRA = 20% basic; DA = 10% basic; overtime = hours × ■150
↓
(RECTANGLE) Gross = basic + HRA + DA + overtime + incentive
↓
(RECTANGLE) PF = 12% basic; tax = 5% of first ■30,000 + 10% of excess
↓
(RECTANGLE) Net salary = gross – PF – tax
↓
(PARALLELOGRAM) Display salary statement
↓
(OVAL) END

How the program works

Gross salary includes basic pay, HRA, DA, overtime and incentive. PF is calculated only on basic pay. Tax is progressive: the first ■30,000 is taxed at 5%, and only the portion above ■30,000 is taxed at 10%.

Python source code

```
basic = float(input("Enter basic pay: Rs. "))
overtime_hours = float(input("Enter overtime hours: "))
incentive = float(input("Enter incentive: Rs. "))

if basic < 0 or overtime_hours < 0 or incentive < 0:
```

```
    print("Invalid input")
else:
    hra = basic * 0.20
    da = basic * 0.10
    overtime_pay = overtime_hours * 150
    gross = basic + hra + da + overtime_pay + incentive
    pf = basic * 0.12
    tax = min(gross, 30000) * 0.05 + max(gross - 30000, 0) * 0.10
    net = gross - pf - tax

    print("\n--- Salary Statement ---")
    print("HRA: Rs.", round(hra, 2))
    print("DA: Rs.", round(da, 2))
    print("Overtime pay: Rs.", round(overtime_pay, 2))
    print("Gross salary: Rs.", round(gross, 2))
    print("PF deduction: Rs.", round(pf, 2))
    print("Tax deduction: Rs.", round(tax, 2))
    print("Net salary: Rs.", round(net, 2))
```

Sample output

```
Enter basic pay: Rs. 25000
Enter overtime hours: 5
Enter incentive: Rs. 1000

--- Salary Statement ---
HRA: Rs. 5000.0
DA: Rs. 2500.0
Overtime pay: Rs. 750.0
Gross salary: Rs. 34250.0
PF deduction: Rs. 3000.0
Tax deduction: Rs. 1925.0
Net salary: Rs. 29325.0
```

Test cases to understand

- Basic ■25,000; overtime 5 hours; incentive ■1,000: gross ■34,250; PF ■3,000; tax ■1,925; net ■29,325.
- Basic ■20,000; no overtime and no incentive: overtime pay ■0.
- Negative basic pay: invalid input.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

5. Multi-Plan Telecommunications Cost Optimization

Problem statement

Compare mobile plans using expected data usage, required validity and budget. Include extra-data charges and choose the lowest-cost plan that meets the requirements.

Objectives

• Calculate the total cost of each plan.
• Check validity and budget.
• Choose the cheapest eligible plan.

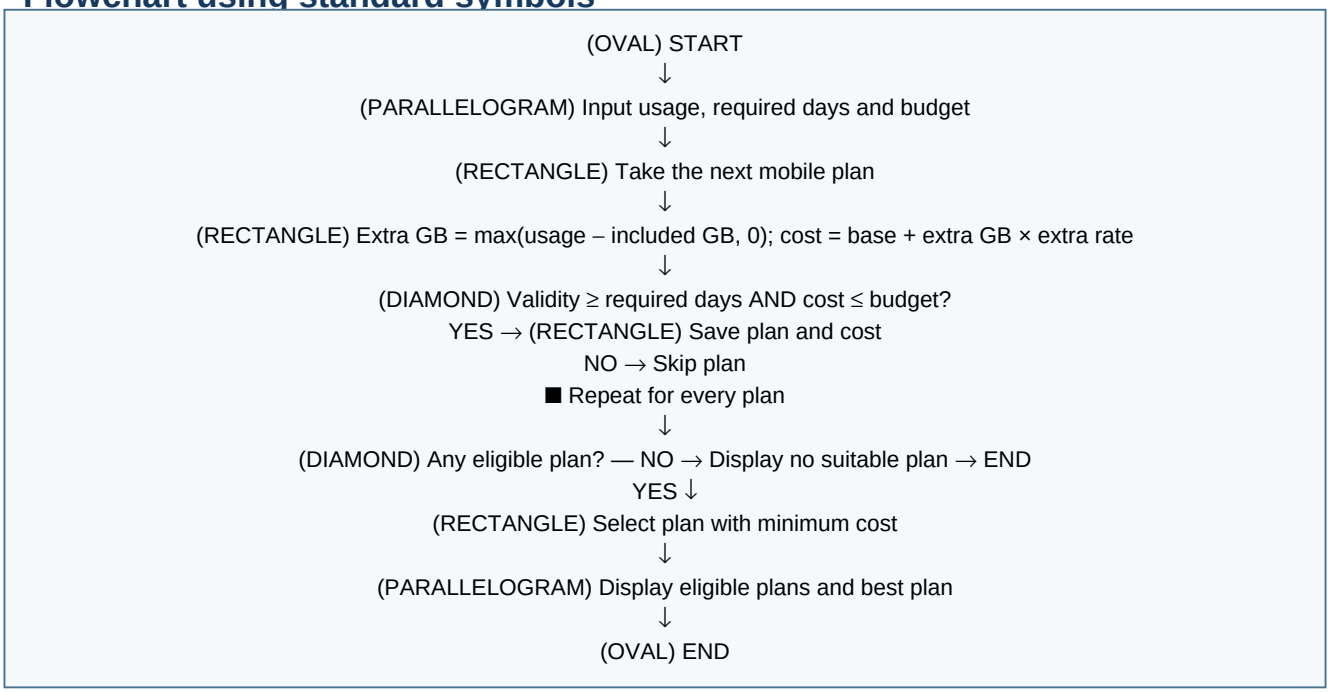
Assumptions / rules

• A plan qualifies only if its validity is at least the required days and its total cost is within budget.
• Each GB above the included data allowance is charged at the plan's excess rate.
• Plan tuple fields: name, base price, included GB, excess price per GB, validity days.

Algorithm (step by step)

1. Read expected data usage, required validity and budget.
2. For each plan, calculate extra GB and total cost.
3. Check whether validity and budget conditions are met.
4. Store each eligible plan with its cost.
5. Choose the eligible plan with the minimum cost.
6. Display eligible plans and the best plan, or display no match.
7. Stop.

Flowchart using standard symbols



How the program works

For each plan, only data above the included allowance creates an extra charge. A plan must pass both the validity and budget checks. Among the plans that pass, the smallest total cost is selected.

Python source code

```
plans = [
    ("Plan A", 299, 10, 20, 28),
    ("Plan B", 399, 20, 0, 56),
    ("Plan C", 249, 5, 20, 28)
```

```
]

usage = float(input("Enter expected data usage (GB): "))
required_days = int(input("Enter required validity (days): "))
budget = float(input("Enter budget: Rs. "))

eligible = []

for name, base, included, extra_rate, validity in plans:
    extra_gb = max(usage - included, 0)
    total_cost = base + extra_gb * extra_rate
    if validity >= required_days and total_cost <= budget:
        eligible.append((name, total_cost, validity))

for plan in eligible:
    print(plan[0], "- Total cost: Rs.", round(plan[1], 2),
          "- Validity:", plan[2], "days")

if eligible:
    best = min(eligible, key=lambda item: item[1])
    print("Most economical plan:", best[0],
          "at Rs.", round(best[1], 2))
else:
    print("No plan satisfies the requirements")
```

Sample output

```
Enter expected data usage (GB): 12
Enter required validity (days): 28
Enter budget: Rs. 500
Plan A - Total cost: Rs. 339.0 - Validity: 28 days
Plan B - Total cost: Rs. 399.0 - Validity: 56 days
Plan C - Total cost: Rs. 389.0 - Validity: 28 days
Most economical plan: Plan A at Rs. 339.0
```

Test cases to understand

- Usage 12 GB, 28 days, budget ■500: Plan A ■339; Plan B ■399; Plan C ■389; choose Plan A.
- Usage 12 GB, 40 days, budget ■500: only Plan B meets validity.
- Usage 12 GB, 28 days, budget ■200: no suitable plan.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

6. Railway Fare and Reservation Calculator

Problem statement

Calculate railway fare using travel distance, travel class, reservation fee and an age-based concession.

Objectives

• Calculate distance fare using slabs.
• Apply class multiplier and senior concession.
• Validate inputs and display final fare.

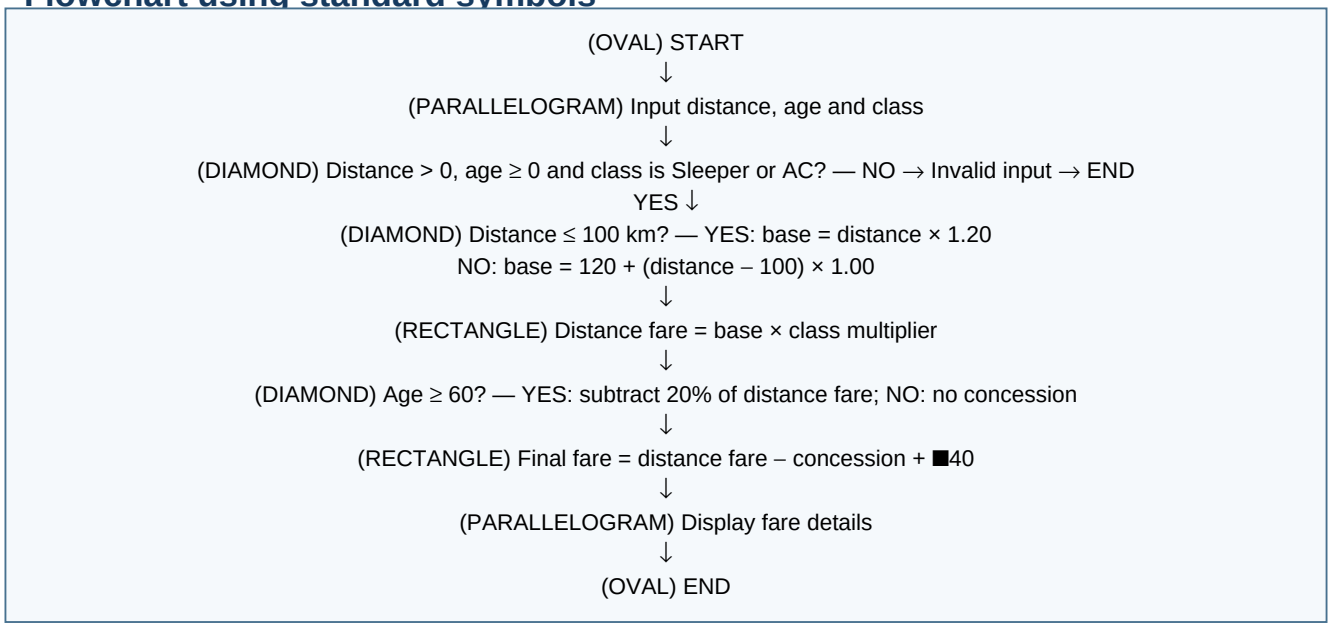
Assumptions / rules

• First 100 km: ■1.20 per km; each additional km: ■1.00.
• Sleeper multiplier = 1.0; AC multiplier = 1.8.
• Reservation fee = ■40 per passenger.
• Age 60 or above receives 20% concession on distance fare only.
• Distance must be positive; age must be non-negative.

Algorithm (step by step)

1. Read distance, age and travel class.
2. Validate distance, age and class.
3. Calculate base fare using the distance slabs.
4. Multiply by the travel-class factor.
5. If age ≥ 60, subtract 20% of distance fare.
6. Add ■40 reservation fee.
7. Display fare details and stop.

Flowchart using standard symbols



How the program works

The distance fare is calculated first. The class multiplier changes that fare. Senior concession is applied to the distance fare only; the ■40 reservation fee is added after the concession.

Python source code

```
distance = float(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))
travel_class = input("Enter class (Sleeper/AC): ").strip().lower()

if distance <= 0 or age < 0 or travel_class not in ("sleeper", "ac"):
```

```
    print("Invalid input")
else:
    if distance <= 100:
        base_fare = distance * 1.20
    else:
        base_fare = 100 * 1.20 + (distance - 100) * 1.00

    multiplier = 1.0 if travel_class == "sleeper" else 1.8
    distance_fare = base_fare * multiplier
    concession = distance_fare * 0.20 if age >= 60 else 0
    final_fare = distance_fare - concession + 40

    print("Distance fare: Rs.", round(distance_fare, 2))
    print("Concession: Rs.", round(concession, 2))
    print("Reservation fee: Rs. 40")
    print("Final fare: Rs.", round(final_fare, 2))
```

Sample output

```
Enter distance in km: 250
Enter passenger age: 65
Enter class (Sleeper/AC): AC
Distance fare: Rs. 486.0
Concession: Rs. 97.2
Reservation fee: Rs. 40
Final fare: Rs. 428.8
```

Test cases to understand

• 250 km, age 65, AC: distance fare ■486; concession ■97.20; final ■428.80.
• 80 km, age 25, Sleeper: distance fare ■96; final ■136.
• First Class: invalid class.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

7. Retail Billing with Multiple Discount Rules

Problem statement

Prepare an itemized invoice for products with different tax rates. Apply a threshold discount with a maximum limit, then calculate tax after discount.

Objectives

• Calculate item subtotals and total subtotal.
• Apply the discount threshold and cap.
• Distribute discount proportionally and calculate tax by product category.

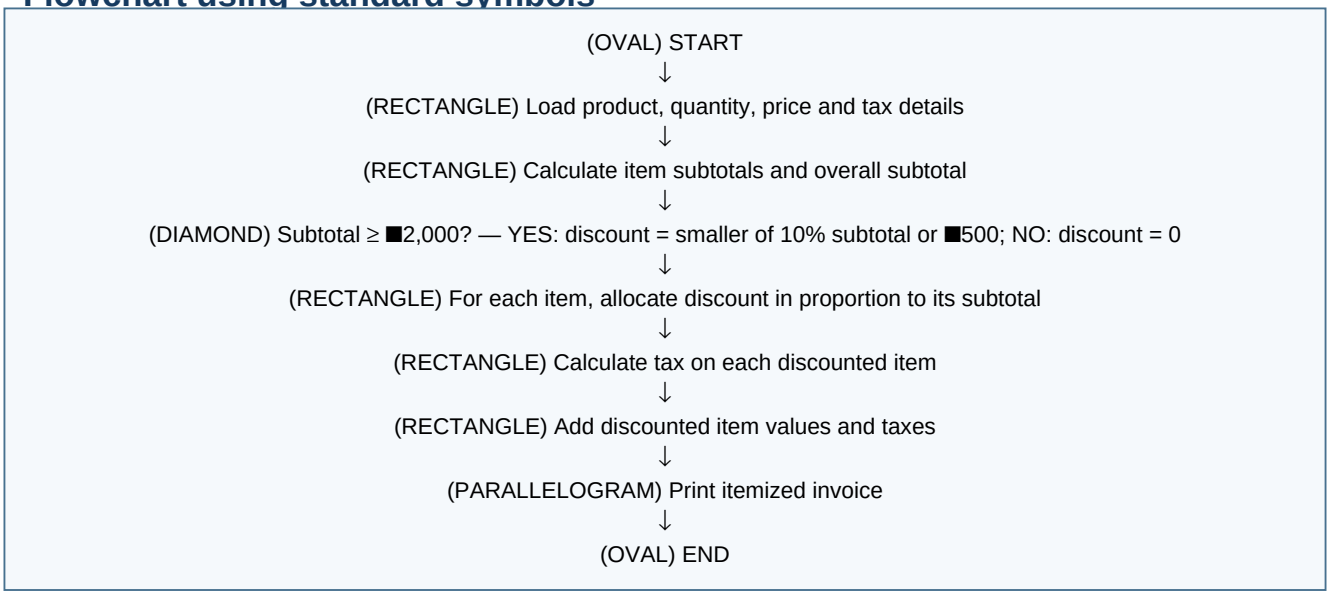
Assumptions / rules

• Notebook: ■100 each with 5% tax; Bag: ■300 each with 12% tax.
• A 10% discount applies when subtotal is at least ■2,000.
• Maximum discount = ■500.
• Discount is allocated proportionally before tax.

Algorithm (step by step)

1. Store each product's name, quantity, unit price and tax rate.
2. Calculate each item subtotal and the total subtotal.
3. If subtotal ≥ ■2,000, discount is 10% of subtotal, limited to ■500; otherwise zero.
4. Allocate the discount according to each item's share of subtotal.
5. Calculate tax on each discounted item.
6. Add discounted values and taxes to get amount payable.
7. Display invoice and stop.

Flowchart using standard symbols



How the program works

The discount is based on the whole invoice subtotal. The cap prevents the discount from exceeding ■500. Each item receives a share of the discount based on its percentage of the subtotal. Tax is then calculated on the reduced item value.

Python source code

```
items = [
    ("Notebook", 10, 100, 0.05),
    ("Bag", 5, 300, 0.12)
]
```

```
subtotal = sum(qty * price for name, qty, price, rate in items)
discount = min(subtotal * 0.10, 500) if subtotal >= 2000 else 0
total_tax = 0

print("--- Itemized Invoice ---")
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))

for name, qty, price, rate in items:
    item_subtotal = qty * price
    share = item_subtotal / subtotal if subtotal else 0
    discounted = item_subtotal - discount * share
    tax = discounted * rate
    total_tax += tax
    print(name, "| Qty:", qty, "| Discounted value: Rs.",
          round(discounted, 2), "| Tax: Rs.", round(tax, 2))

print("Total tax: Rs.", round(total_tax, 2))
print("Amount payable: Rs.", round(subtotal - discount + total_tax, 2))
```

Sample output

```
--- Itemized Invoice ---
Subtotal: Rs. 2500
Discount: Rs. 250.0
Notebook | Qty: 10 | Discounted value: Rs. 900.0 | Tax: Rs. 45.0
Bag | Qty: 5 | Discounted value: Rs. 1350.0 | Tax: Rs. 162.0
Total tax: Rs. 207.0
Amount payable: Rs. 2457.0
```

Test cases to understand

- 10 notebooks at ■100 and 5 bags at ■300: subtotal ■2,500; discount ■250; tax ■207; payable ■2,457.
- 1 notebook and 1 bag: subtotal ■400; no discount.
- Subtotal ■7,000: discount is capped at ■500.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

8. Manufacturing Cost and Profitability Analysis

Problem statement

Calculate manufacturing cost from materials, labour, machine cost and material wastage. Compare the profit with a target profit.

Objectives

• Calculate material and wastage costs.
• Find total production cost, revenue and profit.
• Check whether the target profit is achieved.

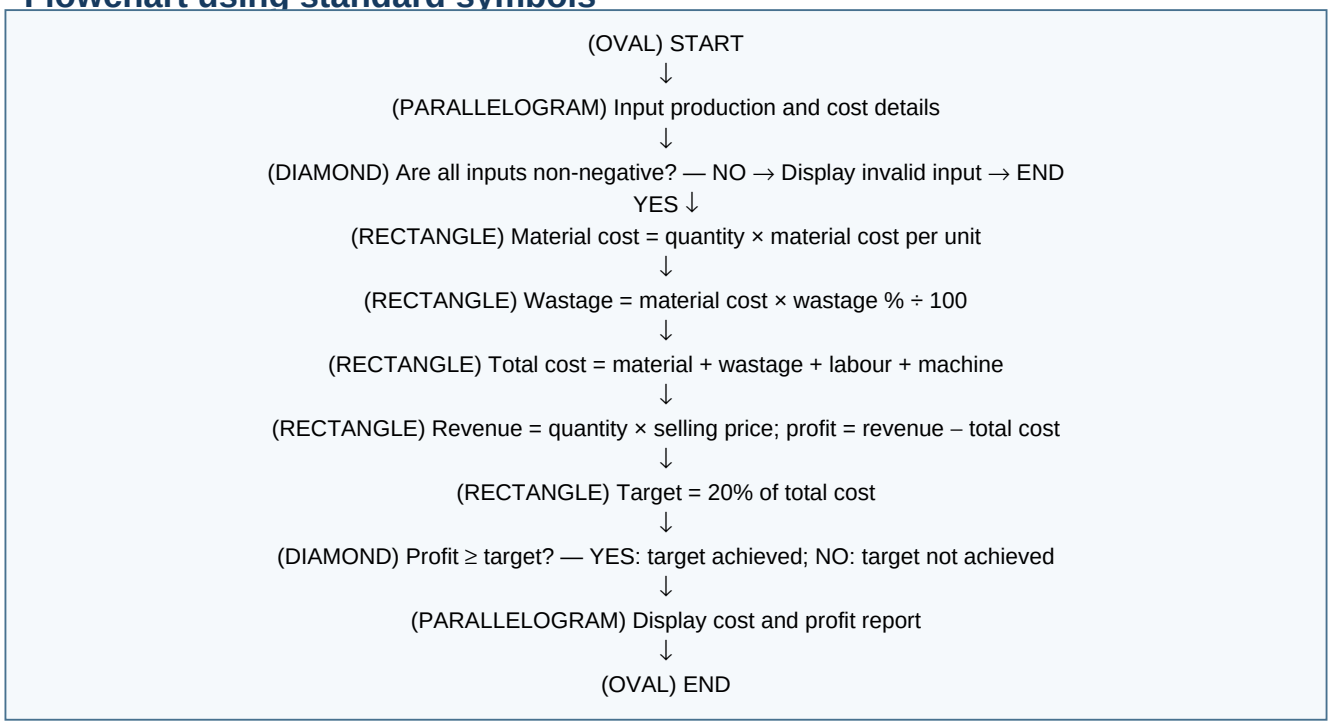
Assumptions / rules

• Material cost = quantity × material cost per unit.
• Wastage cost is a percentage of material cost.
• Total cost = material + wastage + labour + machine cost.
• Target profit = 20% of total production cost.

Algorithm (step by step)

1. Read quantity, material cost per unit, labour, machine cost, wastage percentage and selling price per unit.
2. If any input is negative, display invalid input.
3. Calculate material cost and wastage cost.
4. Calculate total production cost and sales revenue.
5. Profit = revenue – total cost.
6. Target = 20% of total production cost.
7. Compare profit with target and display the result.
8. Stop.

Flowchart using standard symbols



How the program works

Material cost depends on production quantity. Wastage is calculated as a percentage of material cost. Profit is sales revenue minus total production cost. The program checks whether that profit is at least 20% of total cost.

Python source code

```
quantity = int(input("Enter units produced: "))
material_unit = float(input("Material cost per unit: Rs. "))
labour = float(input("Total labour cost: Rs. "))
machine = float(input("Machine cost: Rs. "))
wastage_percent = float(input("Wastage percentage: "))
selling_unit = float(input("Selling price per unit: Rs. "))

values = [quantity, material_unit, labour, machine,
          wastage_percent, selling_unit]

if any(value < 0 for value in values):
    print("Invalid input")
else:
    material_cost = quantity * material_unit
    wastage_cost = material_cost * wastage_percent / 100
    total_cost = material_cost + wastage_cost + labour + machine
    revenue = quantity * selling_unit
    profit = revenue - total_cost
    target = total_cost * 0.20

    print("Material cost: Rs.", round(material_cost, 2))
    print("Wastage cost: Rs.", round(wastage_cost, 2))
    print("Total production cost: Rs.", round(total_cost, 2))
    print("Sales revenue: Rs.", round(revenue, 2))
    print("Profit: Rs.", round(profit, 2))
    print("Target profit: Rs.", round(target, 2))
    print("Target achieved" if profit >= target else "Target not achieved")
```

Sample output

```
Enter units produced: 100
Material cost per unit: Rs. 20
Total labour cost: Rs. 800
Machine cost: Rs. 500
Wastage percentage: 5
Selling price per unit: Rs. 45
Material cost: Rs. 2000.0
Wastage cost: Rs. 100.0
Total production cost: Rs. 3400.0
Sales revenue: Rs. 4500.0
Profit: Rs. 1100.0
Target profit: Rs. 680.0
Target achieved
```

Test cases to understand

• 100 units; material ■20/unit; labour ■800; machine ■500; wastage 5%; selling ■45/unit: total cost ■3,400; revenue ■4,500; profit ■1,100; target achieved.
• 10 units; material ■100/unit; no other costs; selling ■50/unit: loss; target not achieved.
• Negative quantity: invalid input.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

9. Multi-Currency Travel Budget Analysis

Problem statement

Convert travel expenses from USD to Indian rupees using an exchange rate and conversion fee. Compare total spending with a budget and identify the largest expense category.

Objectives

• Convert expenses into INR.
• Include the conversion fee.
• Calculate total spending, budget balance and largest category.

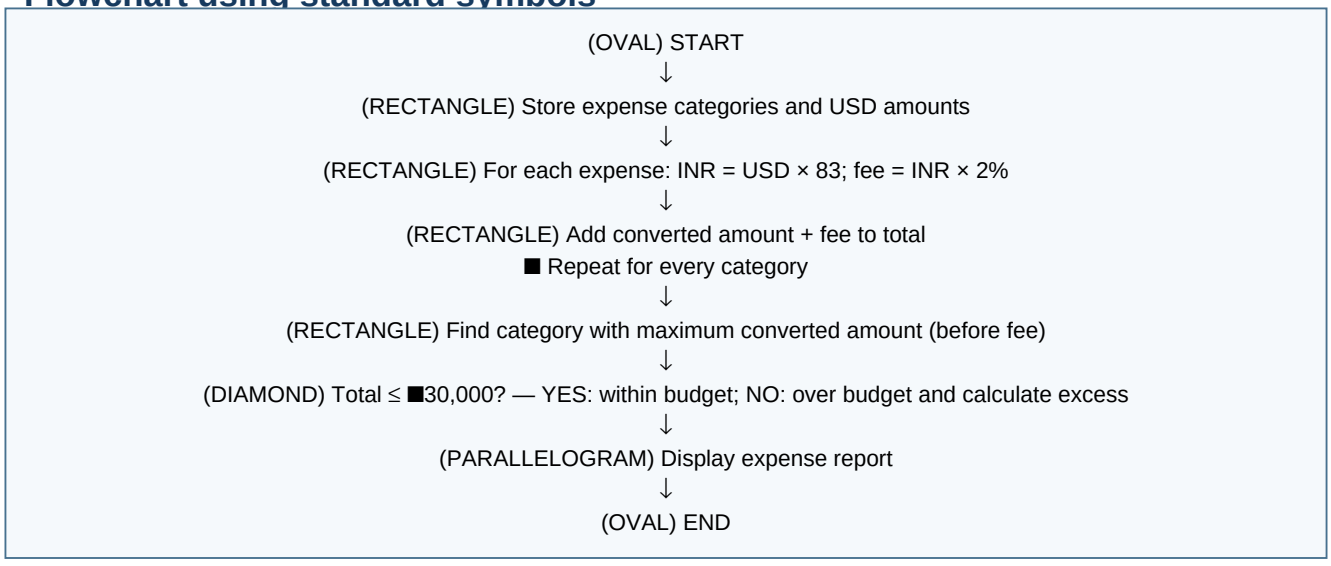
Assumptions / rules

• All expenses are in USD.
• Exchange rate = ■83 per USD.
• Conversion fee = 2% of converted amount.
• Budget = ■30,000.

Algorithm (step by step)

1. Store expenses by category.
2. For each expense, convert USD to INR using 83.
3. Calculate a 2% conversion fee.
4. Add converted amount and fee to total spending.
5. Identify the category with the largest converted amount before fee.
6. Compare total spending with ■30,000.
7. Display total, largest category and budget status; stop.

Flowchart using standard symbols



How the program works

Each USD expense is multiplied by ■83 to convert it into rupees. A 2% fee is added to each converted expense. The largest category is selected using the converted amount before the fee. Total spending, including fees, is compared with the ■30,000 budget.

Python source code

```
expenses = [
    ("Hotel", 200),
    ("Food", 80),
    ("Local transport", 50)
]
```

```
exchange_rate = 83
fee_rate = 0.02
budget = 30000
total = 0
converted_expenses = []

for category, usd in expenses:
    converted = usd * exchange_rate
    fee = converted * fee_rate
    amount_with_fee = converted + fee
    total += amount_with_fee
    converted_expenses.append((category, converted, fee))
    print(category, "- Converted: Rs.", round(converted, 2),
          "| Fee: Rs.", round(fee, 2))

largest = max(converted_expenses, key=lambda item: item[1])
print("Total including fees: Rs.", round(total, 2))
print("Largest expense category:", largest[0])

if total <= budget:
    print("Budget status: Within budget")
else:
    print("Budget status: Over budget by Rs.", round(total - budget, 2))
```

Sample output

```
Hotel - Converted: Rs. 16600 | Fee: Rs. 332.0
Food - Converted: Rs. 6640 | Fee: Rs. 132.8
Local transport - Converted: Rs. 4150 | Fee: Rs. 83.0
Total including fees: Rs. 27937.8
Largest expense category: Hotel
Budget status: Within budget
```

Test cases to understand

- Hotel $200, food $80, transport $50: total including fees ■27,937.80; largest Hotel; within ■30,000.
- Hotel $300, food $100, transport $100: over budget; excess displayed.
- All categories $0: total ■0.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

10. Campus Event Financial Planning

Problem statement

Estimate campus event finances by calculating fixed and per-participant costs, registration income, sponsorship, financial balance and break-even registrations.

Objectives

• Calculate total event cost and income.
• Find surplus or shortfall.
• Calculate the minimum registrations needed to break even.

Assumptions / rules

• Fixed costs: venue ■12,000, equipment ■5,000, transport ■3,000.
• Variable cost per participant: catering ■150 and certificate ■20.
• Registration fee = ■500 per participant.
• Sponsorship = ■10,000.
• Each additional registration contributes ■500 – ■170 = ■330 toward fixed costs.

Algorithm (step by step)

1. Read expected participants.
2. If participant count is negative, display invalid count.
3. Calculate fixed costs and variable cost per participant.
4. Calculate total event cost and total income.
5. Balance = income – cost; positive means surplus, negative means shortfall.
6. Break-even registrations = ceiling((fixed cost – sponsorship) ÷ contribution per participant).
7. Display report and stop.

Flowchart using standard symbols

(OVAL) START
↓
(PARALLELOGRAM) Input expected participants
↓
(DIAMOND) Participants ≥ 0? — NO → Display invalid participant count → END
YES ↓
(RECTANGLE) Fixed cost = 12,000 + 5,000 + 3,000 = ■20,000
↓
(RECTANGLE) Variable cost/person = 150 + 20 = ■170
↓
(RECTANGLE) Total cost = fixed cost + participants × 170
↓
(RECTANGLE) Income = sponsorship 10,000 + participants × 500
↓
(RECTANGLE) Balance = income – total cost
↓
(RECTANGLE) Break-even = ceiling((20,000 – 10,000) ÷ (500 – 170))
↓
(DIAMOND) Balance ≥ 0? — YES: Surplus; NO: Shortfall
↓
(PARALLELOGRAM) Display event report
↓
(OVAL) END

How the program works

Fixed costs do not depend on the number of participants. Variable costs increase by ■170 for every participant. Each registration brings ■500, so the contribution toward fixed costs is ■330. The break-even calculation rounds up because

registrations must be a whole number.

Python source code

```
import math

participants = int(input("Enter expected participants: "))

if participants < 0:
    print("Invalid participant count")
else:
    venue = 12000
    equipment = 5000
    transport = 3000
    catering_per_person = 150
    certificate_per_person = 20
    registration_fee = 500
    sponsorship = 10000

    fixed_cost = venue + equipment + transport
    variable_per_person = catering_per_person + certificate_per_person
    total_cost = fixed_cost + participants * variable_per_person
    income = sponsorship + participants * registration_fee
    balance = income - total_cost
    contribution = registration_fee - variable_per_person
    break_even = math.ceil((fixed_cost - sponsorship) / contribution)

    print("Fixed cost: Rs.", fixed_cost)
    print("Total event cost: Rs.", total_cost)
    print("Total income: Rs.", income)
    print("Financial balance: Rs.", balance)
    print("Break-even registrations:", break_even)

    if balance >= 0:
        print("Event status: Surplus")
    else:
        print("Event status: Shortfall")
```

Sample output

```
Enter expected participants: 50
Fixed cost: Rs. 20000
Total event cost: Rs. 28500
Total income: Rs. 35000
Financial balance: Rs. 6500
Break-even registrations: 31
Event status: Surplus
```

Test cases to understand

• 50 participants: cost ■28,500; income ■35,000; surplus ■6,500; break-even 31.
• 31 participants: cost ■25,270; income ■25,500; surplus ■230.
• –1 participant: invalid participant count.

Conclusion

This case study applies Python input, calculations, conditions and output statements to solve the stated real-world problem. The flowchart shows the order of steps and the decisions made by the program.

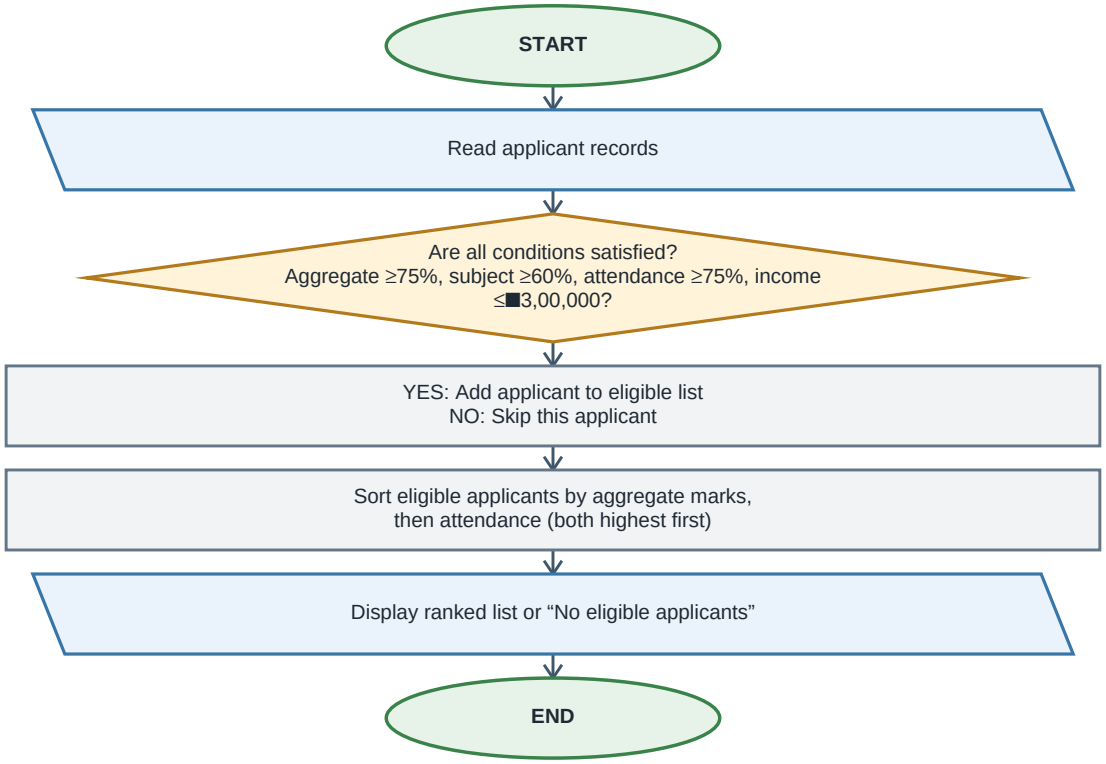
Final submission checklist

1. Run each program in IDLE, VS Code or another Python 3 environment.
2. Compare the actual output with the test cases.
3. Replace sample console illustrations with your own screenshots if your faculty requires actual screenshots.
4. Add your name, roll number, branch and semester.
5. Upload the final material to the GitHub repository named **Python-Activity-Based-Learning**, as stated in the supplied assignment.

Important note: This PDF explains the content of the supplied activity document. Check the code and output in your own Python environment before submission.

1. University Scholarship Allocation

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

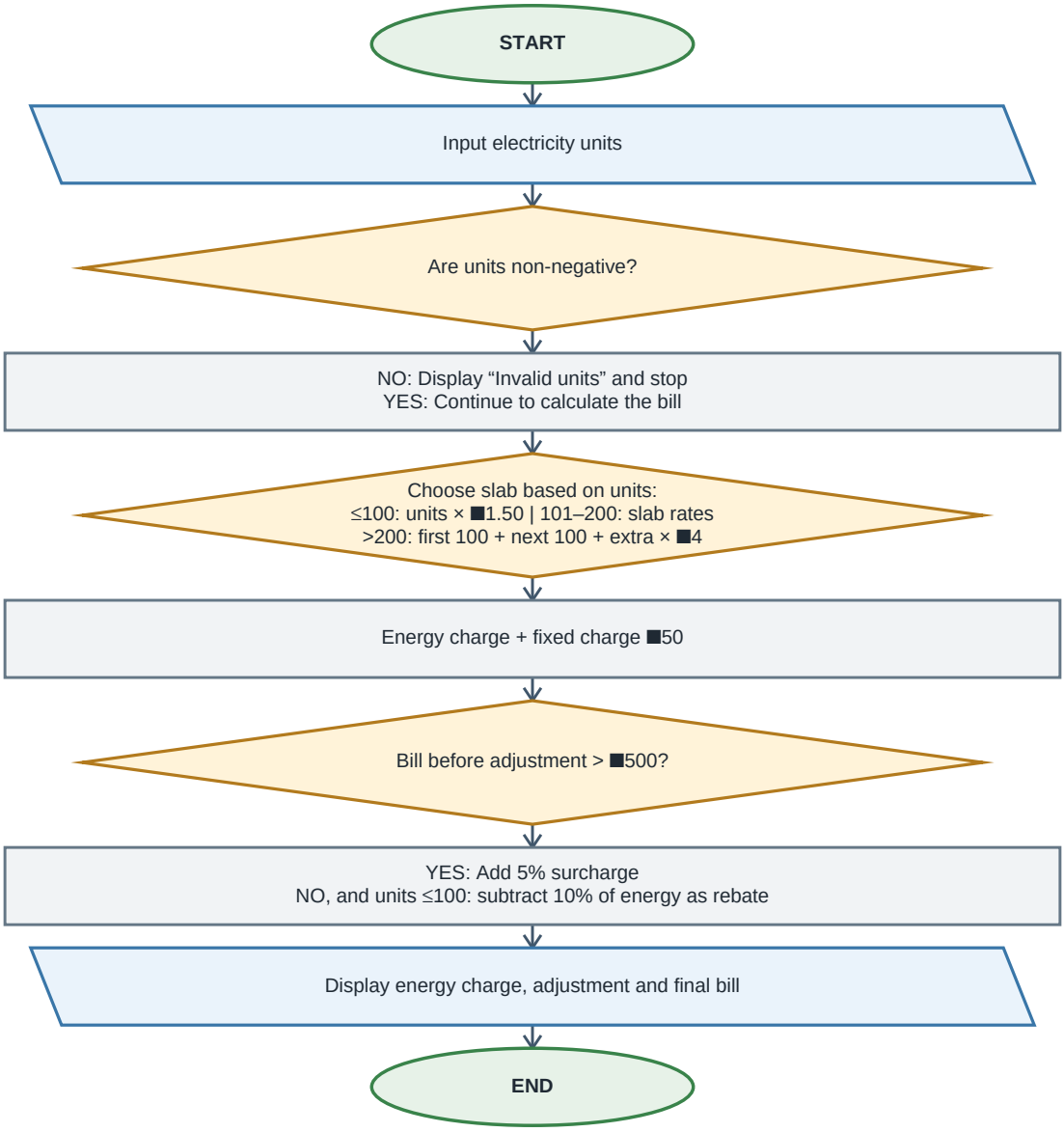


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

2. Progressive Electricity Billing

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

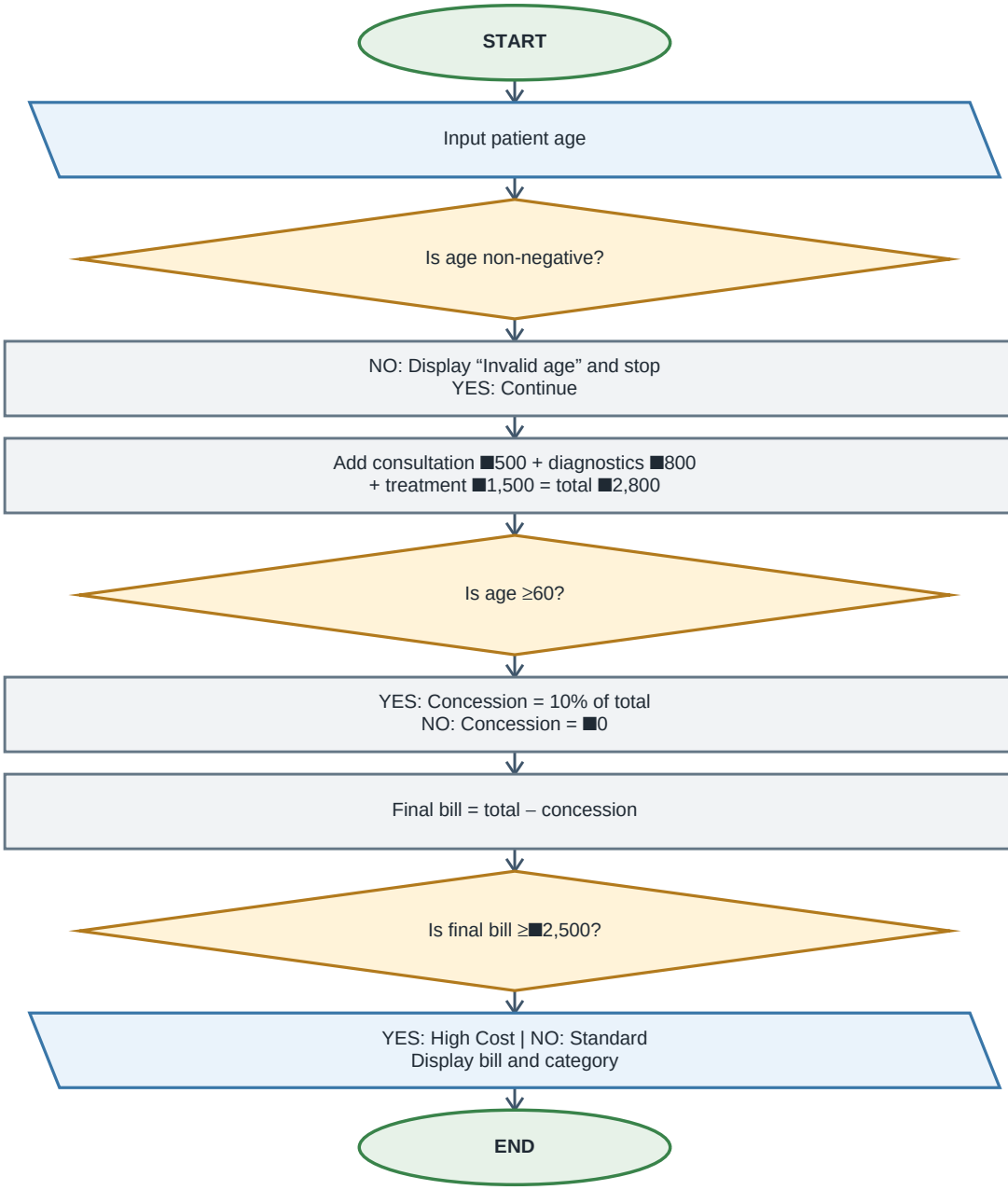


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

3. Hospital Billing and Patient Classification

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

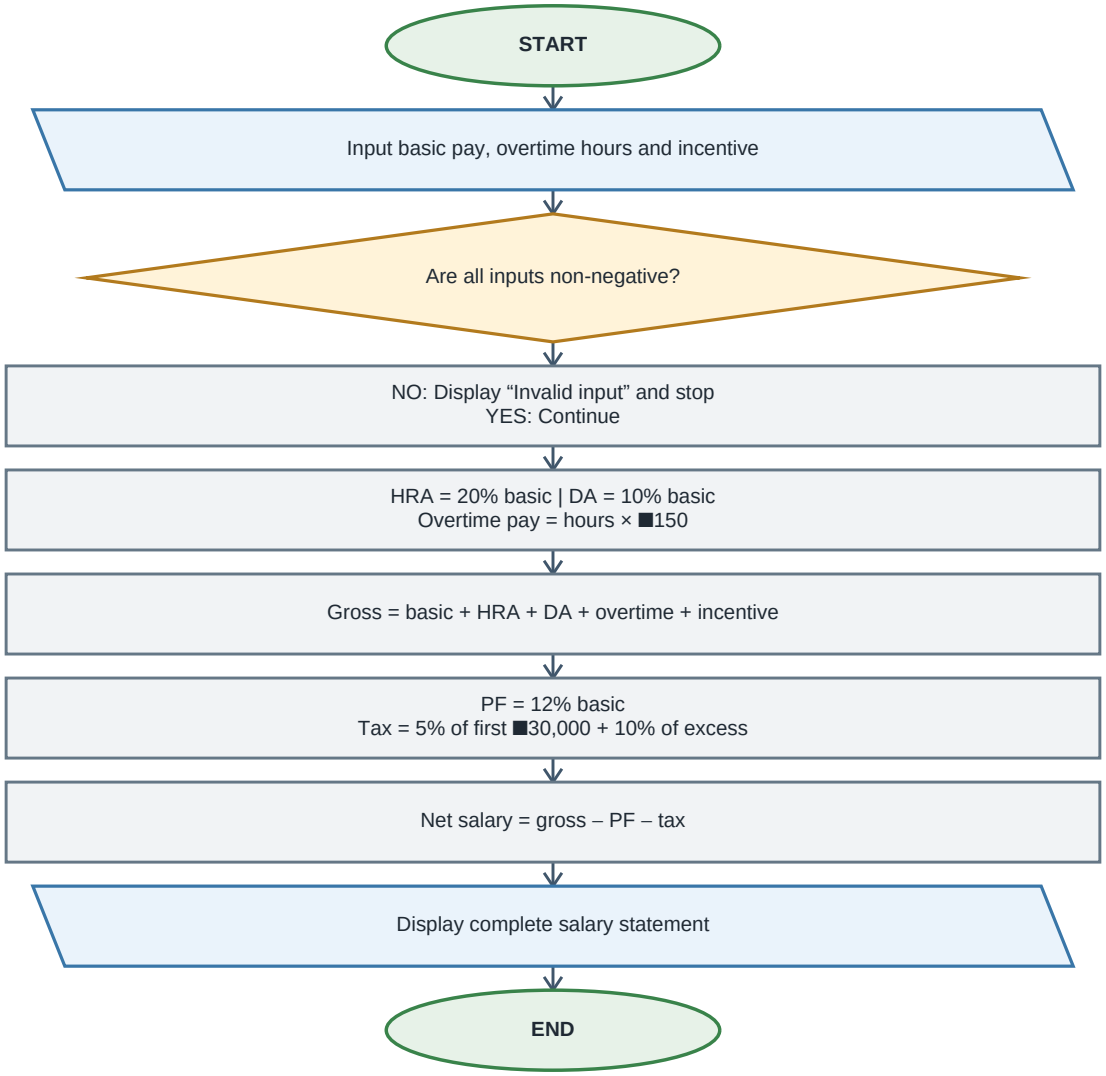


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

4. Employee Payroll and Deduction Analysis

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

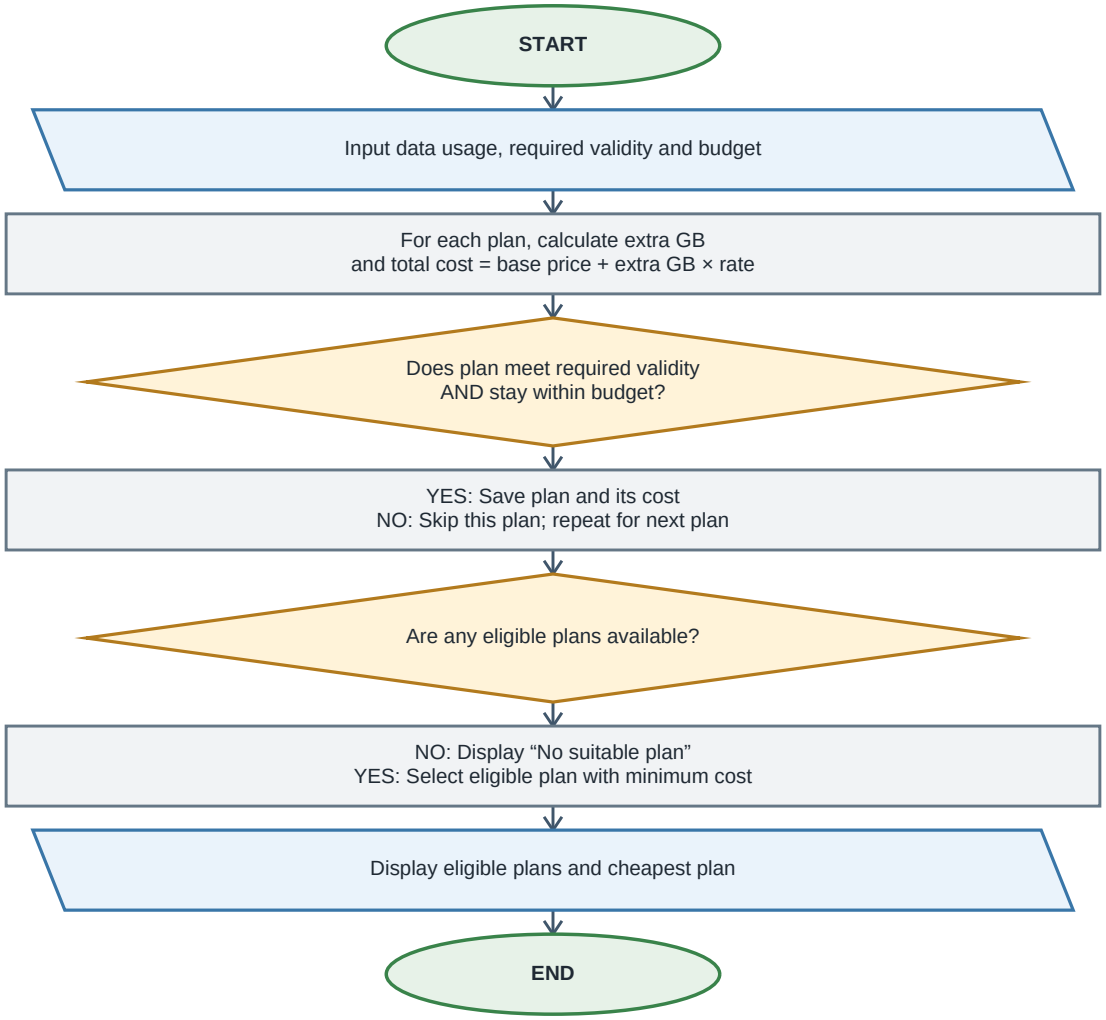


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

5. Multi-Plan Telecommunications Cost Optimization

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

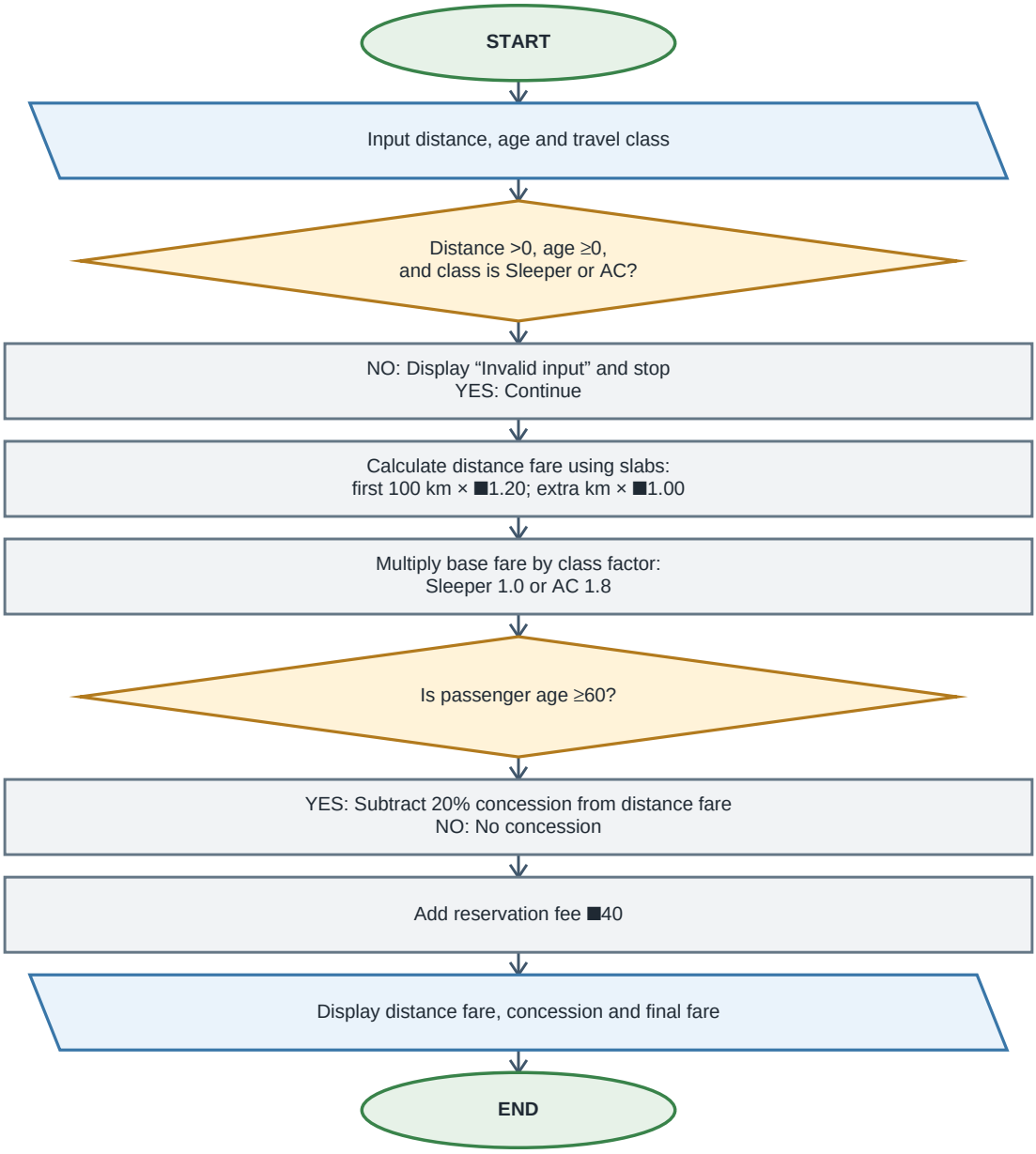


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

6. Railway Fare and Reservation Calculator

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

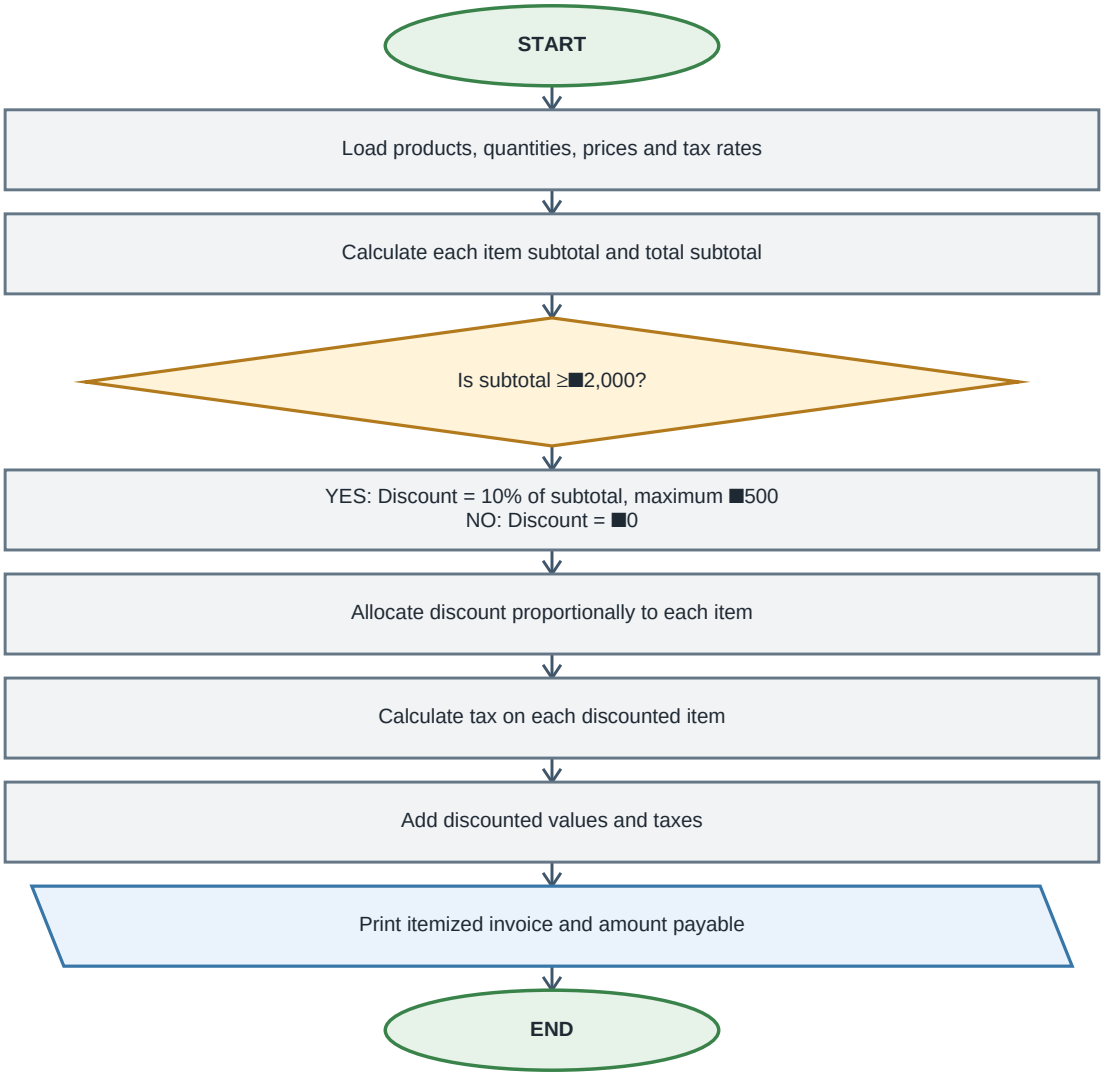


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

7. Retail Billing with Multiple Discount Rules

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

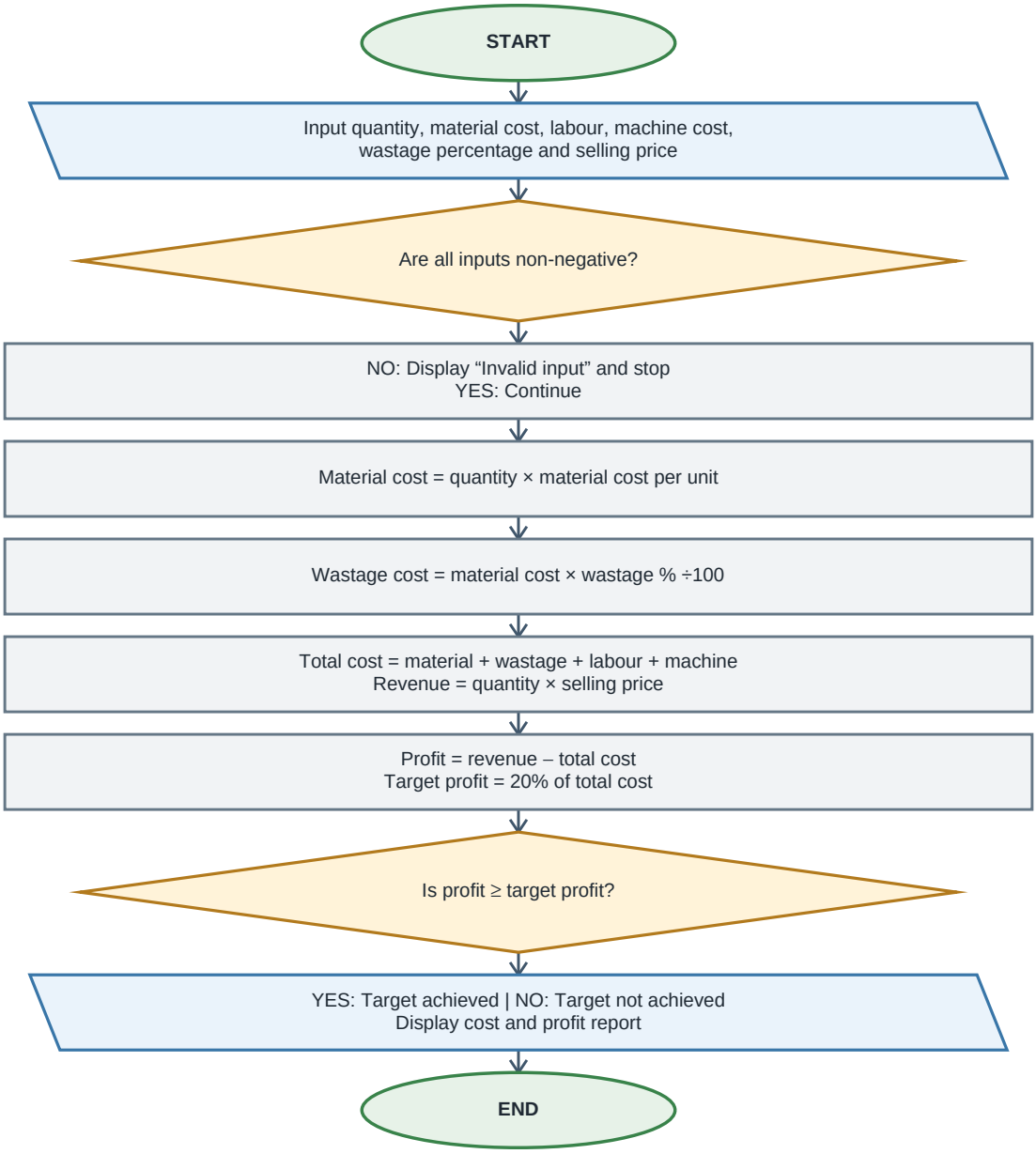


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

8. Manufacturing Cost and Profitability Analysis

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

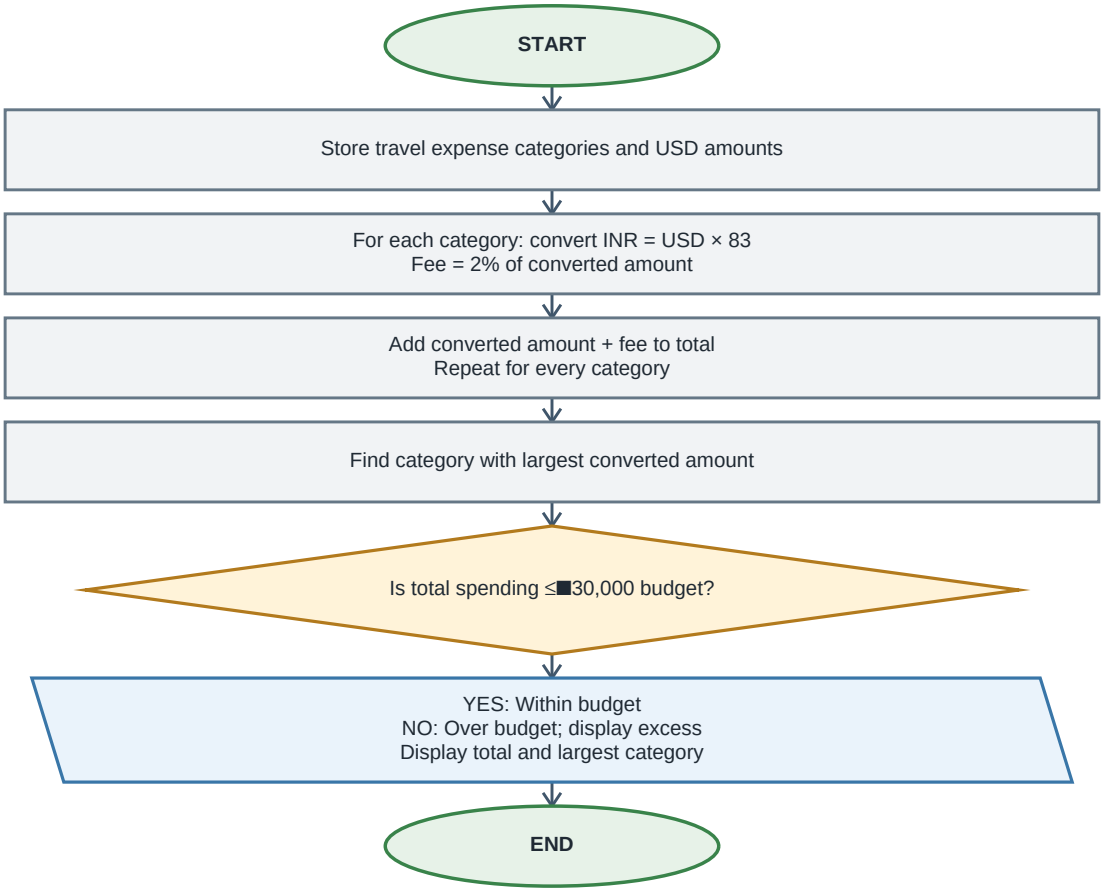


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

9. Multi-Currency Travel Budget Analysis

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds

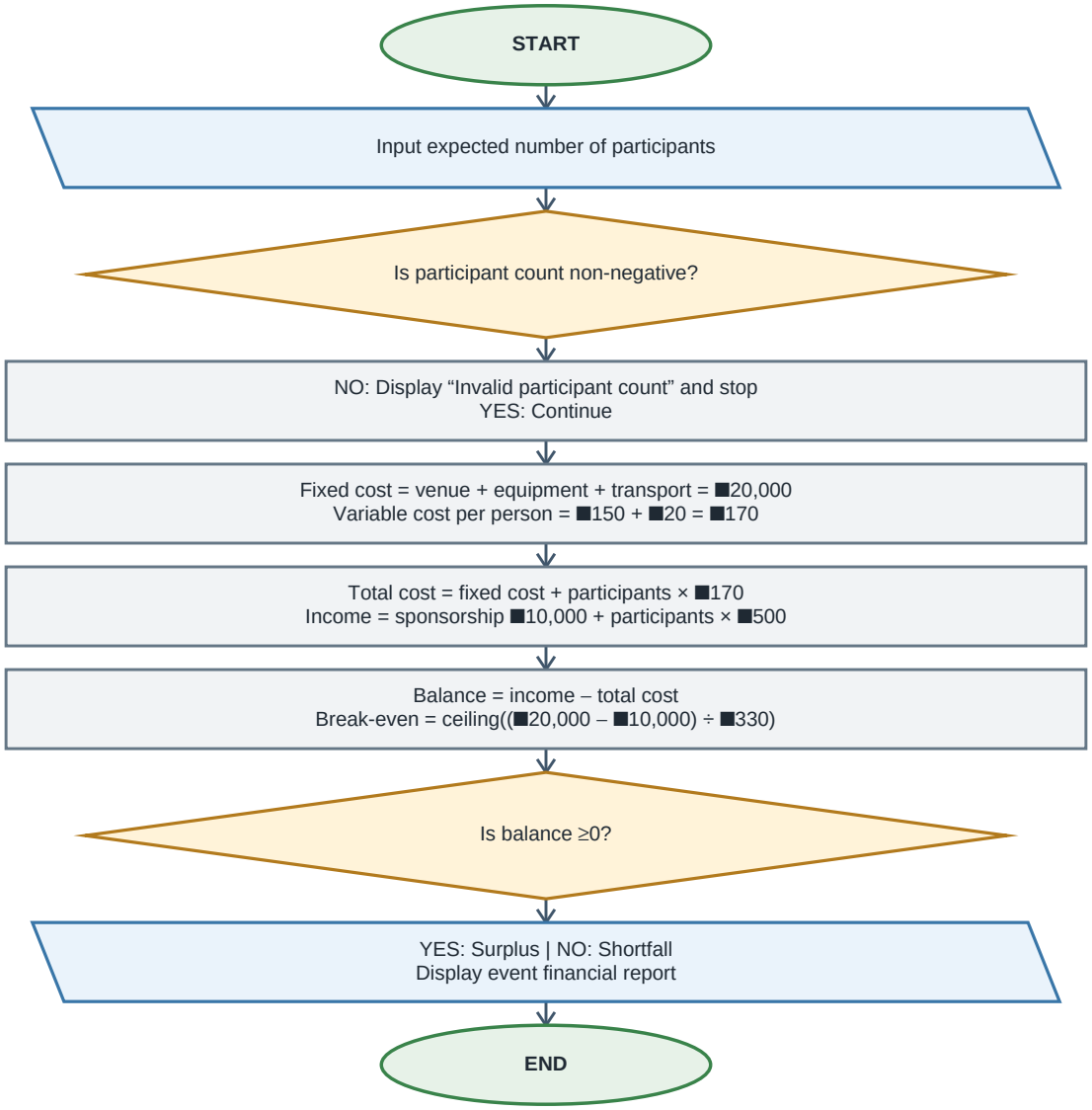


Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols

10. Campus Event Financial Planning

Actual flowchart drawing — follow arrows; read YES/NO at decision diamonds



Oval = Start/End Parallelogram = Input/Output Rectangle = Process Diamond = Decision

Flowchart symbols